Penetration Test Report

by

Hasanur Rahman

engagebranding.com

Target Site: engagebranding.com

Server:

Server technologies: PHP

Table of Contents

Executive Summary **03**

OWASP Top 10 ..... **03**

Criteria for Risk Ratings **05**

Issues By Risk Factor **05**

High-Risk Findings **06**

H1. Sensitive Data Exposure 06

H2. OPIE off-by-one stack overflow 07

Medium-Risk Findings **09**

M1. WebCachePoisoning 09

Low-Risk Findings **12**

L1. Strict transport security not enforced 12

Conclusion **14**

Executive Summary:

The primary goal of this web application penetration testing project was to identify any potential areas related to the application in your current state and to determine how much the system could be compromised by an attacker with certain skills and motivations. It has been evaluated according to the "Best-in-Class" method as defined by the Entrance Test Guidance for the Open Web Application Security Project (OWASP).

OWASP Top 10

The table below summarizes the findings for OWASP's Top 10 list for web application vulnerabilities. OWASP Top 10 represents the list of the most critical web application security flaws, which are accompanied by OWASP security experts from around the world. The list provides a powerful awareness document for web application security and is utilized within many security standards:

1. **Injection**
   Injection flaws, such as SQL, NoSQL, OS, and LDAP injection, occur when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without proper authorization.
2. **Broken Authentication**
   Application functions related to authentication and session management are often implemented incorrectly, allowing attackers to compromise passwords, keys, or session tokens, or to exploit other implementation flaws to assume other users' identities temporarily or permanently.
3. **Sensitive Data Exposure**
   Many web applications and APIs do not properly protect sensitive data, such as financial, healthcare, and PII. Attackers may steal or modify such weakly protected data to conduct credit card fraud, identity theft, or other crimes. Sensitive data may be compromised without extra protection, such as encryption at rest or in transit, and requires special precautions when exchanged with the browser.

4. **XML External Entities (XXE)**
   Many older or poorly configured XML processors evaluate external entity references within XML documents. External entities can be used to disclose internal files using the file URI handler, internal file shares, internal port scanning, remote code execution, and denial of service attacks.
5. **Broken Access Control**
   Restrictions on what authenticated users are allowed to do are often not properly enforced. Attackers can exploit these flaws to access unauthorized functionality and/or data, such as

access other users' accounts, view sensitive files, modify other users' data, change access rights, etc.

6. **Security Misconfiguration**

   Security misconfiguration is the most commonly seen issue. This is commonly a result of insecure default configurations, incomplete or ad hoc configurations, open cloud storage, misconfigured HTTP headers, and verbose error messages containing sensitive information. Not only must all operating systems, frameworks, libraries, and applications be securely configured, but they must be patched/upgraded in a timely fashion.

7. **Cross-Site Scripting (XSS)**

   XSS flaws occur whenever an application includes untrusted data in a new web page without proper validation or escaping, or updates an existing web page with user-supplied data using a browser API that can create HTML or JavaScript. XSS allows attackers to execute scripts in the victim's browser which can hijack user sessions, deface websites, or redirect the user to malicious sites.

8. **Insecure Deserialization**

   Insecure deserialization often leads to remote code execution. Even if deserialization flaws do not result in remote code execution, they can be used to perform attacks, including replay attacks, injection attacks, and privilege escalation attacks.

9. **Using Components with Known Vulnerabilities**

   Components, such as libraries, frameworks, and other software modules, run with the same privileges as the application. If a vulnerable component is exploited, such an attack can facilitate serious data loss or server takeover. Applications and APIs using components with known vulnerabilities may undermine application defenses and enable various attacks and impacts.

10. **Insufficient Logging & Monitoring**

    Insufficient logging and monitoring, coupled with missing or ineffective integration with incident response, allows attackers to further attack systems, maintain persistence, pivot to more systems, and tamper, extract or destroy data. Most breach studies show time to detect a breach is over 200 days, typically detected by external parties rather than internal processes or monitoring.

Criteria for Risk Ratings

The table below outlines the general rules for assigning risk ratings to identified vulnerabilities:

Risk Rating	Description
HIGH	These issues identify conditions that could directly result in the compromise or unauthorized access of a network, system, application, or sensitive information. Examples of High-Risk issues include remote execution of commands, known buffer overflows; unauthorized access, and disclosure of sensitive information.
MEDIUM	These issues identify conditions that do not immediately or directly result in the compromise or unauthorized access of a network, system, application of information but do provide a capability or information that could, in combination with other capabilities or information, result in the compromise or unauthorized access of a network, application or information. Examples of Medium Risk issues include directory browsing, partial access to files on the system; disclosure of security mechanisms, and unauthorized use of services.
LOW	These issues identify conditions that do not immediately or directly result in compromise of a network, system, application, or information but do provide information that could be used in combination with other information to gain insight into how to compromise or gain unauthorized access to a network, system, application or information.
INFORMATIONAL	The issue has been fixed since the previous round of penetration testing.

Issues By Risk Factor

High	2
Medium	1
Low	1

High-Risk Findings

H1. Sensitive Data Exposure

Risk Rating: High

Affected Functionality:

- https://engagebranding.com/info.php

Description:

Sensitive Data Exposure occurs when an organization unknowingly exposes sensitive data or when a security incident leads to the accidental or unlawful destruction, loss, alteration, or unauthorized disclosure of, or access to sensitive data. Such Data exposure may occur as a result of inadequate protection of a database, misconfigurations when bringing up new instances of datastores, inappropriate usage of data systems, and more.

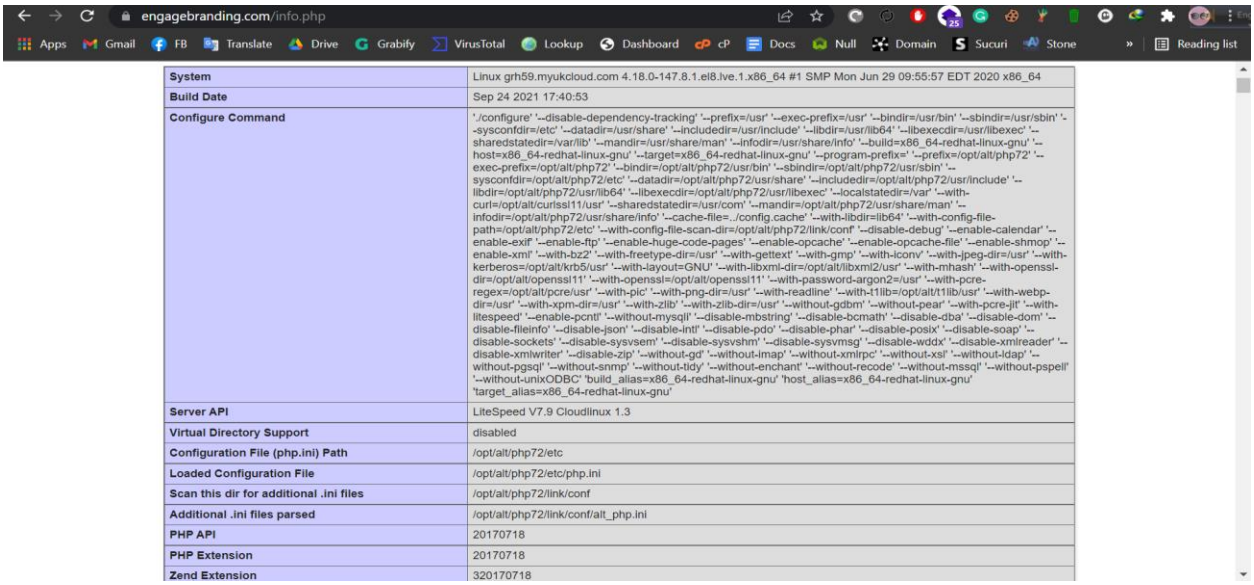
Impact:

Possible sensitive information disclosure.

Solution:

Restrict this link to public visit.

Proof of Concept:



System	Linux grh59.myukcloud.com 4.18.0-147.8.1.el8.lve.1.x86_64 #1 SMP Mon Jun 29 09:55:57 EDT 2020 x86_64
Build Date	Sep 24 2021 17:40:53
Configure Command	'./configure' '--disable-dependency-tracking' '--prefix=/usr' '--exec-prefix=/usr' '--bindir=/usr/bin' '--sbindir=/usr/sbin' '--sysconfdir=/etc' '--datadir=/usr/share' '--includedir=/usr/include' '--libdir=/usr/lib64' '--libexecdir=/usr/libexec' '--sharedstatedir=/var/lib' '--mandir=/usr/share/man' '--infodir=/usr/share/info' '--build=x86_64-redhat-linux-gnu' '--host=x86_64-redhat-linux-gnu' '--target=x86_64-redhat-linux-gnu' '--program-prefix=' '--prefix=/opt/alt/php72' '--exec-prefix=/opt/alt/php72' '--bindir=/opt/alt/php72/usr/bin' '--sbindir=/opt/alt/php72/usr/sbin' '--sysconfdir=/opt/alt/php72/etc' '--datadir=/opt/alt/php72/usr/share' '--includedir=/opt/alt/php72/usr/include' '--libdir=/opt/alt/php72/usr/lib64' '--libexecdir=/opt/alt/php72/usr/libexec' '--localstatedir=/var' '--with-curl=/opt/alt/curlssl11/usr' '--sharedstatedir=/usr/com' '--mandir=/opt/alt/php72/usr/share/man' '--infodir=/opt/alt/php72/usr/share/info' '--cache-file=../config.cache' '--with-libdir=lib64' '--with-config-file-path=/opt/alt/php72/etc' '--with-config-file-scan-dir=/opt/alt/php72/link/conf' '--disable-debug' '--enable-calendar' '--enable-exif' '--enable-ftp' '--enable-huge-code-pages' '--enable-opcache' '--enable-opcache-file' '--enable-shmop' '--enable-xml' '--with-bz2' '--with-freetype-dir=/usr' '--with-gettext' '--with-gmp' '--with-iconv' '--with-jpeg-dir=/usr' '--with-kerberos=/opt/alt/krb5/usr' '--with-layout=GNU' '--with-libxml-dir=/opt/alt/libxml2/usr' '--with-mhash' '--with-openssl-dir=/opt/alt/openssl11' '--with-openssl=/opt/alt/openssl11' '--with-password-argon2=/usr' '--with-pcre-regex=/opt/alt/pcre/usr' '--with-pic' '--with-png-dir=/usr' '--with-readline' '--with-t1lib=/opt/alt/t1lib/usr' '--with-webp-dir=/usr' '--with-xpm-dir=/usr' '--with-zlib' '--with-zlib-dir=/usr' '--without-gdbm' '--without-pear' '--with-pcre-jit' '--with-litespeed' '--enable-pcntl' '--without-mysqli' '--disable-mbstring' '--disable-bcmath' '--disable-dba' '--disable-dom' '--disable-fileinfo' '--disable-json' '--disable-intl' '--disable-pdo' '--disable-phar' '--disable-posix' '--disable-soap' '--disable-sockets' '--disable-sysvsem' '--disable-sysvshm' '--disable-sysvmsg' '--disable-wddx' '--disable-xmlreader' '--disable-xmlwriter' '--disable-zip' '--without-gd' '--without-imap' '--without-xmlrpc' '--without-xsl' '--without-ldap' '--without-pgsql' '--without-snmp' '--without-tidy' '--without-enchant' '--without-recode' '--without-mssql' '--without-pspell' '--without-unixODBC' 'build_alias=x86_64-redhat-linux-gnu' 'host_alias=x86_64-redhat-linux-gnu' 'target_alias=x86_64-redhat-linux-gnu'
Server API	LiteSpeed V7.9 Cloudlinux 1.3
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/opt/alt/php72/etc
Loaded Configuration File	/opt/alt/php72/etc/php.ini
Scan this dir for additional .ini files	/opt/alt/php72/link/conf
Additional .ini files parsed	/opt/alt/php72/link/conf/alt_php.ini
PHP API	20170718
PHP Extension	20170718
Zend Extension	320170718

H2. OPIE off-by-one stack overflow

Risk Rating: High

Affected Functionality:

PORT STATE SERVICE

21/tcp open FTP

Description:

"One time Passwords In Everything" (OPIE) is a Unix-like login and password package installed on the server and the client which makes untrusted networks safer against password-sniffing packet analysis software. The OPIE package is sometimes used by FTP servers to provide security for an FTP session. A remote code execution vulnerability has been reported in OPIE. The vulnerability is due to an off-by-one vulnerability while processing an attacker-controlled username. The vulnerable code bounds-check the input string but incorrectly limits it to (buffersize+1). An off-by-one error in OPIE library 2.4.1-test1 and earlier, allows remote attackers to cause a denial of service or possibly execute arbitrary code via a long username. Successful exploitation could result in the execution of arbitrary code with root privileges."

Solution:

This protection will detect and block attempts to exploit this vulnerability.

In order for the protection to be activated, update your Security Gateway product to the latest IPS update. For information on how to update IPS, go to SBP-2006-05, click on the Protection tab and select the version of your choice.

Security Gateway R80 / R77 / R75

- In the IPS tab, click Protections and find the Multiple Vendors OPIE Off-By-One Stack Buffer Overflow protection using the Search tool and Edit the protection's settings.
- Install policy on all Security Gateways.

Proof of Concept:

```
PORT      STATE  SERVICE
21/tcp    open   ftp
| ftp-libopie:
|   VULNERABLE:
|   OPIE off-by-one stack overflow
|     State: LIKELY VULNERABLE
|     IDs:  CVE:CVE-2010-1938  BID:40403
|     Risk factor: High  CVSSv2: 9.3 (HIGH) (AV:N/AC:M/Au:N/C:C/I:C/A:C)
|       An off-by-one error in OPIE library 2.4.1-test1 and earlier, allows remote
|       attackers to cause a denial of service or possibly execute arbitrary code
|       via a long username.
|     Disclosure date: 2010-05-27
|     References:
|       http://security.freebsd.org/advisories/FreeBSD-SA-10:05.opie.asc
|       http://site.pi3.com.pl/adv/libopie-adv.txt
|       https://www.securityfocus.com/bid/40403
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2010-1938
```

Medium-Risk Findings

M1. WebCachePoisoning

Risk Rating: Medium

Affected Functionality:

- https://engagebranding.com/info.php

Vulnerable Header:

- X-Custom-Ip-Authorization:
- Cluster-Client-Ip:
- True-Client-Ip:
- Client-Ip:
- X-Wap-Profile:
- X-Forwarded-Host:
- X-Forwarded-Proto:
- X-Proxyuser-Ip:
- Referer:

Etc.

Description:

Web cache poisoning is an advanced technique whereby an attacker exploits the behavior of a web server and cache so that a harmful HTTP response is served to other users. Fundamentally, web cache poisoning involves two phases. First, the attacker must work out how to elicit a response from the back-end server that inadvertently contains some kind of dangerous payload. Once successful, they need to make sure that their response is cached and subsequently served to the intended victims. A poisoned web cache can potentially be a devastating means of distributing numerous different attacks, exploiting vulnerabilities such as XSS, JavaScript injection, open redirection, and so on.

Impact:

A poisoned web cache can potentially be a devastating means of distributing numerous different attacks, exploiting vulnerabilities such as XSS, JavaScript injection, open redirection, and so on.

Solution:

You should also take the following precautions when implementing caching:

- If you are considering excluding something from the cache key for performance reasons, rewrite the request instead.
- Don't accept fat GET requests. Be aware that some third-party technologies may permit this by default.
- Patch client-side vulnerabilities even if they seem unexploitable. Some of these vulnerabilities might actually be exploitable due to unpredictable quirks in your cache's behaviour. It could be a matter of time before someone finds a quirk, whether it be cache-based or otherwise, that makes this vulnerability exploitable.

Proof of Concept:

Request

Pretty | Raw | Hex

```
1  GET /info.php/ HTTP/2
2  Host: engagebranding.com
3  Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/web
   p,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
4  Upgrade-Insecure-Requests: 1
5  Referer:
   https://engagebranding.com/keeping-your-wordpress-health-in-check/
6  Accept-Encoding: gzip, deflate
7  Accept-Language: en-US,en-GB;q=0.9,en;q=0.8
8  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/95.0.4638.69 Safari/537.36
9  X-Custom-Ip-Authorization: TOKEN1337
10 Cache-Control: max-age=0
```

Response

Pretty | Raw | Hex | Render

```
                max-age=0
              </td>
            </tr>
929         <tr>
              <td class="e">
                $_SERVER['HTTP_UPGRADE_INSECURE_REQUESTS']
              </td>
              <td class="v">
                1
              </td>
            </tr>
930         <tr>
              <td class="e">
                $_SERVER['HTTP_X_CUSTOM_IP_AUTHORIZATION']
              </td>
              <td class="v">
                TOKEN1337
              </td>
            </tr>
931         <tr>
              <td class="e">
                $_SERVER['HTTP_X_HTTPS']
              </td>
```

Low-Risk Findings

L1. Strict transport security not enforced

Risk Rating: Low

Affected Functionality:

- http://engagebranding.com/wp-content/plugins/engage-testimonials/css/main.css
- http://engagebranding.com/wp-content/plugins/bdthemes-element-pack/assets/css/bdt-uikit.css
- http://engagebranding.com/wp-content/plugins/jet-engine/assets/css/frontend.css
- http://engagebranding.com/wp-content/plugins/bdthemes-element-pack/assets/css/element-pack-site.css
- http://engagebranding.com/wp-content/plugins/elementor/assets/lib/font-awesome/css/font-awesome.min.css

Description:

The application fails to prevent users from connecting to it over unencrypted connections. An attacker able to modify a legitimate user's network traffic could bypass the application's use of SSL/TLS encryption, and use the application as a platform for attacks against its users. This attack is performed by rewriting HTTPS links as HTTP so that if a targeted user follows a link to the site from an HTTP page, their browser never attempts to use an encrypted connection. The sslstrip tool automates this process.

To exploit this vulnerability, an attacker must be suitably positioned to intercept and modify the victim's network traffic. This scenario typically occurs when a client communicates with the server over an insecure connection such as public Wi-Fi, or a corporate or home network that is shared with a compromised computer. Common defenses such as switched networks are not sufficient to prevent this. An attacker situated in the user's ISP or the application's hosting infrastructure could also perform this attack. Note that an advanced adversary could potentially target any connection made over the Internet's core infrastructure.

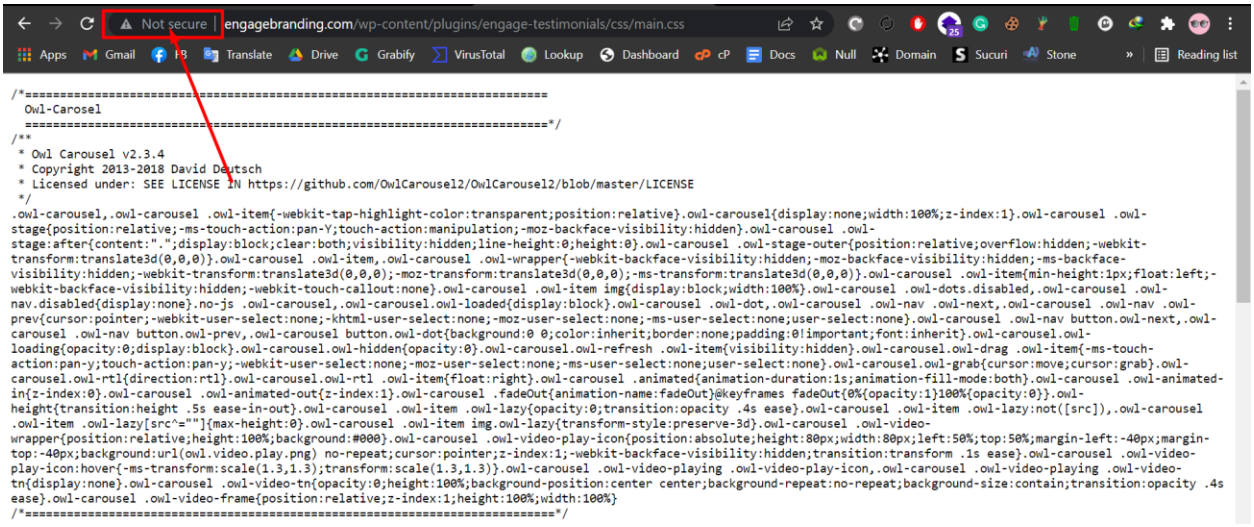
Solution:

The application should instruct web browsers to only access the application using HTTPS. To do this, enable HTTP Strict Transport Security (HSTS) by adding a response header with the name 'Strict-Transport-Security' and the value 'max-age=expireTime', were to expire time is the time in seconds that

browsers should remember that the site should only be accessed using HTTPS. Consider adding the 'includeSubDomains' flag if appropriate.

Note that because HSTS is a "trust on first use" (TOFU) protocol, a user who has never accessed the application will never have seen the HSTS header, and will therefore still be vulnerable to SSL stripping attacks. To mitigate this risk, you can optionally add the 'preload' flag to the HSTS header, and submit the domain for review by browser vendors.

Proof of Concept:



Conclusion

Finally, I have completed the penetration & vulnerability test of the web application. These tests have been based on technology and known threats to date in this document. All safety issues discovered during this exercise have been analyzed and described in this report.

Please note that as technology and risks change over time, so will the vulnerabilities associated with the operation of the systems described in this report, as well as the steps required to reduce the exposure to such vulnerabilities.